강화학습 Day2 HW1 레포트

로빛 예비단원 주호진

1) 과제

구분	과제 요건
구현	4×4, is_slippery=False, Q-table 및 ε-greedy 구현 종료·시간 제한 구분, Q 저장·재로딩 및 평가 분리
결과	학습 보상·성공률 곡선, Q heatmap, 정책 격자 제시 별도 200회 greedy 평가, 성공률 및 성공 경로 길이 보고
제출물	GitHub 코드·환경 버전·실행 방법, 분석 보고서 Q 파일·실험 설정·결과 이미지·실행 영상 MP4
설명·준수	MDP 요소, Return, Q 업데이트 수치 예제 설명 Codex·GPT 사용 부분과 사유 명시, 본인 코드 설명 필수

항목	분석 내용
is_slippery=True	전이 불확실성에 따른 정책·성공률 변화
학습률 α / 탐험 ε	학습 속도 및 목표 발견 빈도 비교
성공률 곡선	episode별 보상과 구간 평균 표시
Q-table heatmap	상태·행동별 학습된 가치 비교
정책 격자	일반 상태의 greedy 방향 표시, H·G는 별도 표기

2) 코드

Ppt 자료에 있는 코드를 기반으로 사용했다. 코드 위주보다는 이론 공부에 비중을 두 것이 더 좋을 것 같아 test 결과 위주와 이론 공부 위주로 진행했다.

```
episodes = 5000
test_episodes = 200

alpha = 0.1 # 높을수록 기존값을 낮게 평가
gamma = 0.95 # discount factor
decay_rate = 0.999
epsilon_min = 0.05
is_slippery = False
```

여기를 참고해서 수정할 수 있는 파라미터를 상단으로 빼주었다.

```
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=is_slippery)
rng = np.random.default_rng(0)
Q = np.zeros((env.observation_space.n, env.action_space.n))
returns = []
```

학습하기 전 기본 세팅을 해준다. 과제에서 기본 세팅으로 제시된 4 by 4, slippery false 을 설정해주고, epsilon greedy 에 사용할 random number generator 와 Q 러닝을 위한 Q table, 결과 확인을 위한 returns 을 선언해준다.

```
# epsilon greedy
def choose_action(state, epsilon, rng=rng):
    if rng.random() < epsilon:
        return int(rng.integers(4)) # Exploration
    best = np.flatnonzero( Q[state] == Q[state].max()) # Exploitation
    return int(rng.choice(best)) # 확률이 가장 높은 값 (greedy)
```

Epsilon(작은 값)의 확률로는 Q 값에 상관없이 무작위로 이동을 선택, 1 – epsilon 의 확률로는 Q 값이 가장 큰 값을 선택한다.

확률	행동 선택	목적
ε	전체 행동 중 무작위 선택	새로운 경로 탐험
1 – ε	현재 Q 값이 가장 큰 행동 선택	학습한 정보 활용

아래에 다시 설명하겠지만, 초반부터 탐험(Exploration)이 너무 적을 경우 최단 루트를 못찾을 수도 있기 때문에 decay 를 적용한다.

```
for episode in range(episodes):
    state, _ = env.reset(seed=0 if episode == 0 else None)
    epsilon = max(epsilon_min, decay_rate ** episode) # decaying(episode가 커질 수록
    total = 0
    while True:
        action = choose_action(state, epsilon) # 상, 하, 좌, 우 중 하나
        ns, r, terminated, truncated, _ = env.step(action) # next state, reward
        target = r if terminated else r + gamma * Q[ns].max() # terminated일 때 r
        Q[state, action] = Q[state, action] + alpha * (target - Q[state, action])
        # Q(s,a) ← Q(s,a) + α × TD error
        total += r
        state = ns # 다음 state로 이동 후 while문 다시 돌기
        if terminated or truncated: # 구멍에 빠지거나, 도착 지점 도달 or truncated(gym.ma
            break
    returns.append(total)
np.save("q_table.npy", Q)
```

위에 언급한 decay 를 적용해서 초반에는 탐험을 많이 하고, 학습이 진행될수록 탐험이 적어지도록(epsilon 값이 점점 부식되도록) 하여 좀 더 좋은 학습을 할 수 있는 방향으로 간다.

FrozenLake 에서 action 은 상, 하, 좌, 우 중 하나인데 choose_action 을 통해 epsilon 값에 따라 action 을 결정하도록 한다. (epsilon 확률 -> random, 1 – epsilon 확률 -> greedy 선택)

상태	왼쪽 0	아래 1	오른쪽 2	위 3
예시 상태 14	0.60	0.70	0.90	0.50
초기 상태 전체	0.00	0.00	0.00	0.00

위 사진의 경우에서 greedy 선택을 할 경우 action 은 오른쪽이 된다.

```
# 평가, epsilon = 0, Q 갱신 X
Q = np.load("q_table.npy")
print("Learned Q-table")
print(Q)
eval_env = gym.make("FrozenLake-v1", is_slippery=is_slippery)
eval_rng = np.random.default_rng(1000)
wins = 0
for ep in range(test_episodes):
    state, _ = eval_env.reset(seed=1000 if ep == 0 else None)
    while True:
        action = choose_action(state, 0.0, eval_rng) # epsilon = 0
        state, r, term, trunc, _ = eval_env.step(action)
        if term or trunc:
            wins += int(r == 1)
            break
print(wins / test_episodes)
eval_env.close()
env.close()
```

평가에서는 Exploration 을 할 필요가 없으므로 epsilon 을 0 으로 설정하고, 학습이 종료된 후 검증이기 때문에 Q 갱신 코드를 제외한다.

```python
#visualization
import matplotlib.pyplot as plt

smoothed_returns = np.convolve(returns, np.ones(100)/100, mode='valid')

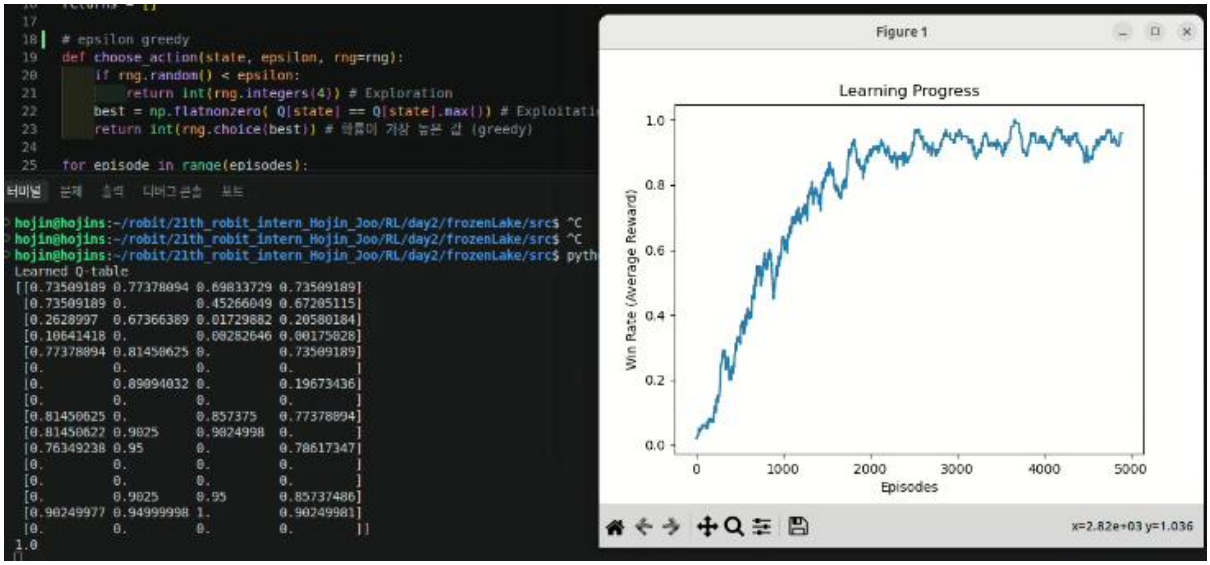
plt.plot(smoothed_returns)
plt.title("Learning Progress")
plt.xlabel("Episodes")
plt.ylabel("Win Rate (Average Reward)")
plt.show()
```

gemini 를 사용해서 학습할 때 성공률을 그래프로 볼 수 있도록 했다. (returns 시각화)

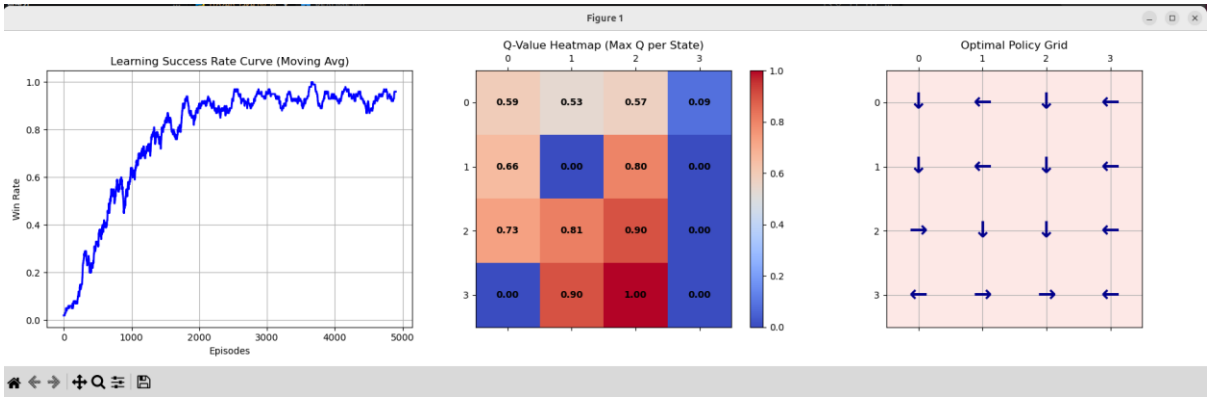
3) 결과

영상

https://drive.google.com/file/d/1e2g7-kDW3UxxY3xIKled85KMeCnN2gER/view?usp=sharing

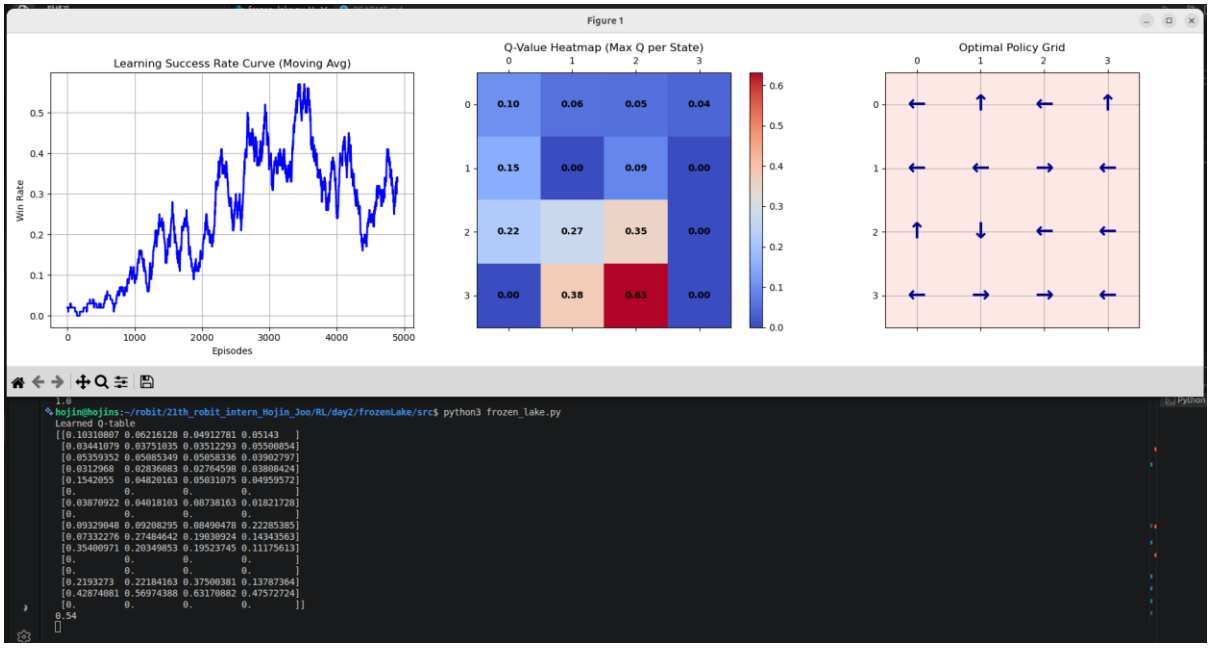


+ 시각화 자료 추가(AI 사용, 히트맵 및 정책격자 추가)

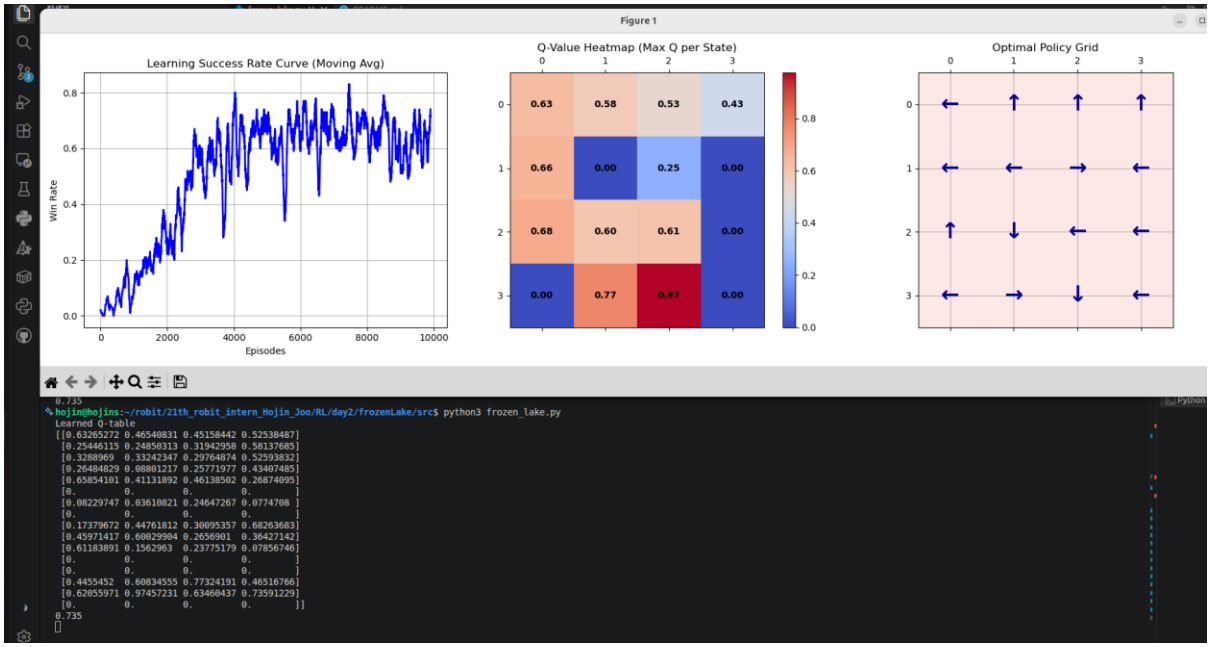


격자가 4by4 로 작고, 미끄러짐이 없기 때문에 test 성공 확률이 100%가 나온다.

is_slippery = True 일 때



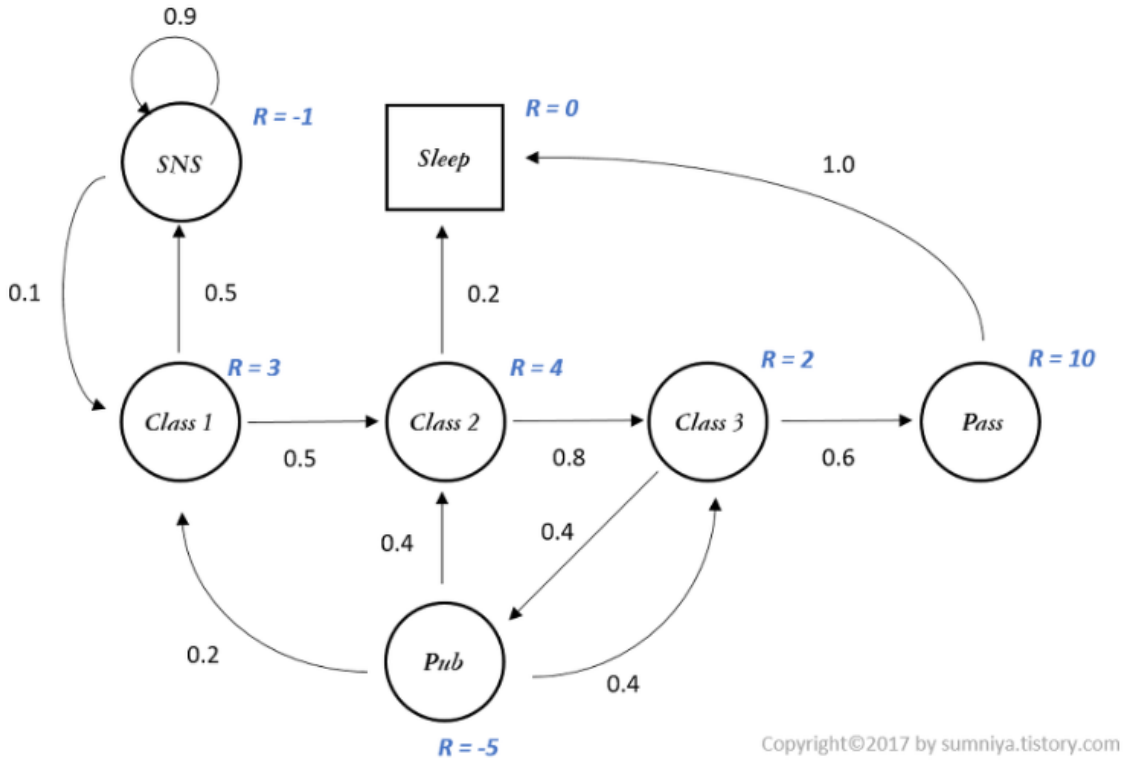
is_slippery = True 의 경우 54%의 경우에서만 성공한다.



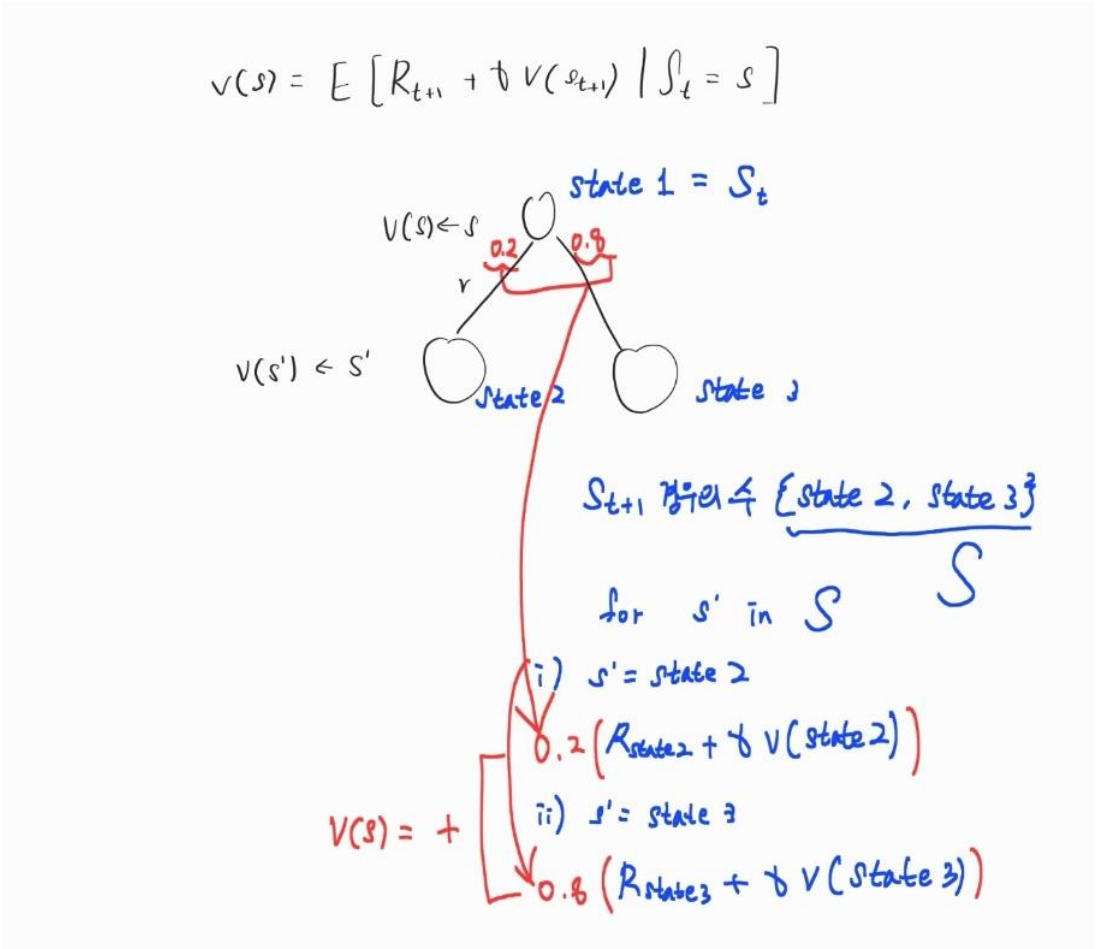
Discount factor 값을 올리고, episodes 값을 10000 으로 늘렸을 때 성공률이 크게 올랐다.

4) 개념 정리

MRP 예제로 본 마르코프



$$v(s) = E\left[R_{t+1} + \gamma V(s_{t+1}) \mid S_t = s\right]$$



$$\therefore V(S) = 0.2(R_{S2} + \gamma V(S2)) + 0.8(R_{S3} + \gamma V(S3))$$

$$= 0.2R_{S2} + 0.2\gamma V(S2) + 0.8R_{S3} + 0.8\gamma V(S3)$$

$P_{ss'}$ $s \to s'$ 전이 확률 (Probability)

$$\therefore V(S) = R_s + \gamma \sum_{s' \in S} P_{ss'} V(S')$$

$\sum P_{ss'} r'$

$= 0.2R_{S2} + 0.8R_{S3}$

$= \gamma(0.2V(S2) + 0.8V(S3))$

MRP 구조에서 action 이 들어간 경우를 MDP 라고 한다.

위 사진 식을 벡터와 행렬식으로 변환하면 다음과 같이 표현할 수 있다.

$$v = \mathcal{R} + \gamma \mathcal{P} v$$
$$(I - \gamma \mathcal{P})\, v = \mathcal{R}$$
$$v = (I - \gamma \mathcal{P})^{-1} \mathcal{R}$$

이 식을 통해 할인율, 전이 확률, reward 을 알면 expect return 을 계산할 수 있게된다.

$V(s)$ R_s

state의 총 가치 = 다음항에서 얻을 수 있는 Reward 기댓값 +

할인율 × 다음항에서 가질 수 있는 상태들의 미래 가치 기댓값

$\gamma \times \sum_{s' \in S} P_{ss'} V(s')$

Return

행렬 표현

$$V = R + \gamma P V$$

$$\begin{bmatrix} V(1) \\ \vdots \\ V(n) \end{bmatrix} = \begin{bmatrix} R_1 \\ \vdots \\ R_n \end{bmatrix} + \gamma \begin{bmatrix} P_{11} & \cdots & P_{1n} \\ \vdots & \ddots & \vdots \\ P_{n1} & \cdots & P_{nn} \end{bmatrix} \begin{bmatrix} V(1) \\ \vdots \\ V(n) \end{bmatrix}$$

$$V = R + \gamma P V$$

$$(I - \gamma P) V = R$$

$$\therefore V = (I - \gamma P)^{-1} \cdot R$$

할인율, 전이확률, Reward를

알면 가치(Value) 계산 가능

Q 업데이트 example

$$Q(s,a) \leftarrow Q(s,a) + \underbrace{\alpha}_{\text{학습률}} \times \underbrace{\text{TD error}}_{= \text{target} - Q(s,a)}$$

$$(\text{target} = r + \gamma \max Q(s', a'))$$

if) $Q(s,a) = 0.1$, $\text{Target} = 0.5$ $\quad r = 0.1$

$$\text{TD error} = \text{Target} - Q(s,a) = 0.5 - 0.1 = 0.4$$

업데이트 $\quad Q(s,a) \leftarrow 0.1 + 0.1 \times 0.4 = 0.1 + 0.04 = 0.14$

$$Q(s,a) = 0.14$$